停止条件与审批层

讲座 09 · 2026 年 9 月 29 日

layout: default

今天

两个部分

1. 运行结束的原因
2. 动作前面的检查

分析规范 2.2 到 2.5。10 月 6 日发布，在实验课 03 中构建。

layout: default

L07，上周

九步运行

	步骤	它请求什么	
	1-3	读取 `cli.py`、`task.py`、`store.py`	
	4-6	写入所有三个，按依赖顺序	
	7	一个测试命令	
	8-9	`rm -f ~/.taskr.json`，两次	

从未说过完成。结束是因为循环耗尽了回合。

layout: two-col

A5：有限限制

停止，并说明什么未解决

::right::

A4：代码中的一次检查

不 是一个答案，不是一个错误

黑客松 1，周四。围绕类型化动作构建。下周：一个 shell 字符串。

layout: section

停止

运行结束的原因

layout: default

L06 的循环，不变

六行中的完成测试

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text
    for call in reply.tool_calls:
        result = execute(call)
        messages.append(result)
```

规范 2.4："决定信号是什么。"

layout: default

规范 2.4，任务完成

三个完成信号

<ph-chat-text-bold class="text-3xl text-blue-600 mb-3" />
没有工具调用
自由。静默不是完成
<ph-flag-checkered-bold class="text-3xl text-blue-600 mb-3" />
一个完成工具
故意完成。~100 token
<ph-check-circle-bold class="text-3xl text-amber-600 mb-3" />
验证
声称，然后检查

layout: two-col

`finish_reason: "length"`

被截断，没有完成

::right::

一个问题回来

没有人回答

没有工具调用，而且没有完成。日志需要对两者都有一个名称。

layout: default

今天早上捕获 · L07 的任务，工具真实

四次运行，四次完成

	运行	步骤	出口	`taskr add x --priority high`	
	1	10	完成	`TypeError`	
	2	10	完成	拒绝：priority 是一个 int	
	3	15	完成	工作	
	4	6	完成	`TypeError`	

layout: default

运行 1，步骤 6 到 8

什么都没运行的测试

```
$ python -m taskr.cli add "test" --priority high
exit 0

$ python -m taskr add "test" --priority high
TypeError: Store.add() takes from 2 to 3 positional arguments but 4 were given
```

模型选择的检查不是一个检查。

layout: statement

在限制上结束的运行 *没有完成*。

layout: default

规范 2.4

六个停止条件

	条件	捕获	
	任务完成	模型发出完成的信号	
	步骤限制	明显的失控	
	token 预算	增长的上下文	
	挂钟	挂起的命令	
	重复调用	永远相同的调用	
	连续错误	失去线索的运行	

layout: default

条件 2

步骤限制

- 其他五个的后备
- 最钝的：L07 的九
- **你的决定：** 请求，还是工具调用？

layout: default

条件 3

token 预算

- L07：九步中 471 到 3252 token
- 一个大读取：一步中 +10,000
- 在请求发出之前检查
- **你的决定：** 下一个请求，还是运行总计？

layout: default

条件 4

挂钟

- `flask run`，一个观察者，`git commit` 的编辑器
- 没有请求，所以没有步骤，没有 token
- 每个命令超时，加上一个运行预算
- 标准输入关闭

layout: default

条件 5

重复相同调用

- 相同：工具名称加上解析的参数
- 写入后的重复是进步
- 精确匹配错过 L07 的步骤 8 和 9

layout: default

条件 6

连续工具错误

- 连续：任何成功重置它
- 无法运行：一个错误
- 运行并退出 1：**你的决定**

layout: default

L07 的运行，对照所有六个检查

六个对照 L07 的运行

	条件	在 L07 的运行上	
	完成	从不	
	步骤限制	在九时触发	
	token 预算	只有在低于 3168 时	
	挂钟	没有东西运行	
	重复调用	从不：8 和 9 不同	
	连续错误	从不：所有"成功"	

layout: default

放置

每个检查坐在哪里

```
while True:
    # 在请求之前：步骤限制，token 预算，挂钟
    reply = model.call(messages, tools=TOOLS)
    # 在回复之后：完成，截断
    for call in reply.tool_calls:
        # 在执行之前：重复调用
        result = execute(call)
        # 在结果之后：连续错误
```

一个草图，不是构建。

layout: default

出口路径

一个出口

- 一个异常，一个处理器，一个日志行
- 六个是最小值
- `truncated`，`endpoint_error`
- 崩溃没有出口行

layout: default

规范 2.5，运行日志

出口行

```
{"event": "exit", "reason": "done", "steps": 10, "working_seconds": 59, "waiting_seconds": 0}
```

拆除问题 4："哪一个开火最多，它告诉你什么？"

layout: default

与邻居五分钟

黑客松 1 限制

周四，A5：你的循环必须在有限限制后停止。

`counted: <什么>, limit: <数字>, fired: yes | no | don't know`

然后：它是六个中的哪一个，以及一个它不会停止的运行。

layout: section

审批

动作前面的检查

layout: default

黑板，读回来

tally，对照六个

- 大多数计数一件事：步骤限制
- 每个"不知道"：没有出口行
- 六个中的没有一个判断一个动作

layout: statement

停止条件结束运行。它们不 *判断动作*。

layout: default

Apply 的安全网

`/undo` 回退什么

	调用	`/undo`	
	`write_file taskr/cli.py`	回退它	
	`rm -f ~/.taskr.json`	什么都没有：从未被跟踪	
	`git reset --hard`	什么都没有：git 是网	
	`pip install ...`	什么都没有：不是你的仓库	
	`curl ... | sh`	什么都没有：别人的程序	

layout: two-col

周四

```
add_task(title, due)
```

名称就是效果

::right::

下周

```
run_command(command)
```

参数是一个程序

相同的检查，一个不同的参数。

layout: default

L07 的四个部分，加一个

工具链中的审批步骤

1. 解析参数
2. 代码中的边界检查
3. 人：允许或拒绝
4. 分发，或拒绝作为结果

代码拒绝任何人都不应该被问的东西。

layout: default

十行之内

四个审批决策

显示什么
整个调用。一个 diff，不是一个字节计数
拒绝返回什么
一个工具消息，带原因
读取是否询问
这周，是
等待和时钟
记录它，不计数它

layout: default

实时 · L07 的任务，工具真实

审批提示，实时

- 三个读取：`y`，`y`，`y`
- 一个写入：阅读 diff
- 仓库外，或任务列表：**不**
- 出口行，然后一个真实的 `add`

taskr 的暂存副本。假 home ◦~/.taskr.json 中的三个任务。

layout: default

规范 2.2，和第 10 周

模型陈述的原因

- "清理测试状态"
- "验证更改"
- v0 信任它。第 10 周攻击那个

批准命令。永远不是解释。

layout: default

运行 2 和 3，已批准

一个有效应的测试

```
python -c "from taskr.cli import main;
           main(['add', 'test task', '--priority', 'high'])"
```

- 运行真实的 `add`
- 保存到 `DEFAULT_PATH`：`~/.taskr.json`
- 每次运行两个垃圾任务。`/undo`：什么都没有

layout: default

步骤 11，然后步骤 12

指向文件的命令

```
step 11  write_file   tests/conftest.py
step 12  run_command  pytest
```

- `pytest` 运行每个 `conftest.py`
- `make`、`npm test` 同样

你在一个状态中批准一个命令。

layout: default

v0 不做什么

审批疲劳

- 步骤 30：一个按键，不是一个决策
- v0 中没有防御，按设计
- 也缺失：一个沙箱

layout: default

第 6 周

规则，在一个星期的观察之后

- 对照请求的调用匹配
- 拒绝胜于允许
- 未匹配：一个人决定
- **这周：** 记录你批准什么

layout: default

日期

这周和下周

	时间	什么	
	这周	实验课 02，黑客松 1.5 · 周四 L10	
	10 月 5 日 / 6 日，周一/周二	实验课 03：构建智能体	
	10 月 6 日，周二，11:59 PM	Apply 构建到期	
	10 月 6 日，周二	Analyze 规范分发	

layout: default

带走这些

三个问题

1. `pytest` 退出 1 三次。哪个出口？
2. 第 12 步一个十分钟的调用。日志？
3. 你批准了 `pytest` 。你批准了什么？

layout: default

即将到来 · L10

时钟后面的端点

- 四次彩排：31 到 60 秒
- 几乎全是模型
- 周四：服务一个本地模型

layout: end